Team Number :	apmcm2300618
Problem Chosen :	A

2023 APMCM summary sheet

Apple picking mainly relies on manual labor. With the development of urbanization, the number of agricultural workers has decreased, leading to the increase of labor costs. As a result, the application of automated apple-picking robots has become an inevitable trend. This article establishes a highly accurate apple recognition and analysis model based on **OpenCV** and **convolutional neural network models**, using **deep learning** and **residual networks**.

For Question 1 and Question 2, this article first carried out light processing, Gaussian filtering, grayscale processing, **watershed construction**, binarization, corrosion and dilation processing on the images in Attachment 1 to remove image noise and achieve efficient recognition of apple. Subsequently, based on the non-uniform irrational B-spline fitting method, improvements were made by using **self-adaptive B-spline curve** to accurately outline the contour of the subject on the preprocessed image. Then, apples were circled for counting, and the number of apples in different images was obtained. The total number of apples was 1657, and a histogram of the distribution of all apples in Attachment 1 was drawn. Subsequently, a coordinate axis was established with the bottom left corner of the image as the origin, and the center of each apple circular box was used as the coordinate point to obtain the position of the apples in each image. A two-dimensional scatter plot of their coordinates was drawn under this circumstance.

For Question 3, this article constructed a **convolutional neural network(CNN)** model. To obtain more training sets, this article used the method of cropping, flipping and rotating, scaling, shifting, Gaussian noise, and color jitter, normalizes them, constructed a convolutional neural network (CNN) model, and added a **Dropout layer** to optimize the model, reducing overfitting risks. Finally, 142 mature apple images and 54 immature apple images were obtained.

For Question 4, based on the preprocessing of Question 1, this article used the **Mason rotation algorithm** to generate random points, and used **ray crossing algorithm** and **Monte Carlo algorithm** to determine whether the random points fall within the contour of the apple. By calculating its probability density function and comparing it with the image area, the two-dimensional area of the apple could be estimated. Then, this article analyzed the relationship between the number of apples in the image and the two-dimensional area of apples. Based on the study of apple density, a histogram of the quality distribution of apples can be obtained.

For Question 5, this article first preprocessed the images and improved the training effect of the model by adjusting the size and normalizing them. Subsequently, the **ResNet-50 residual network model** was constructed in the CNN model, and the concept of residual learning was introduced using inductive transfer learning to effectively reduce the impact of gradient vanishing and exploding in deep neural networks. Based on the ResNet pre-trained model, a fruit classification recognition model was constructed, and after 30 epochs of training, the model quickly converged, achieving an accuracy of over 96%.

Keywords: Apple recognition, OpenCV, Convolutional Neural Network (CNN), residual network

Contents

1 Introduction

1.1 Problem Background

China is the main producer of apples, accounting for more than 50% of the world's apple planting area and yield. With the development of urbanization in China, the aging of agricultural workers is severe, and labor costs have also increased sharply, bringing huge development restrictions to the apple industry, which mainly relies on manual work to complete picking. As a result, the popularization of various apple-picking robots has developed.

However, most of the current picking robots are unable to accurately recognize relevant features such as "fruit obstruction", "leaf obstruction", and "fruit maturity", making inaccurate judgments about the apple picking environment and conditions, and it sometimes lead to significant damage to the fruit. In addition, identifying apples correctly and distinguishing them from other fruits with similar color and shape is also a challenge.

Therefore, establishing an intelligent apple image recognition model and popularizing intelligent picking and classification robots to meet the huge market demand has become a current hot topic.

1.2 Restatement of the Problem

Based on the following analysis on the images of harvest-ready apples in the attachment, we need to solve the following five problems:

- Problem 1: Abstract the features of harvest-ready apples' images in Attachment 1, establish a mathematical model to count the number of apples in each image and draw a distribution diagram of all apples in Attachment 1.
- Problem 2: Based on Attachment 1, distinguish the position of apples in each photo with the lower left corner of the photo as the coordinate origin, draw a two-dimensional scatter plot to illustrate the location of all apples in attachment 1.
- Problem 3: Based on Attachment 1, establish a mathematical model to find out the ripeness of apples in each photo and use a histogram to illustrate the ripeness distribution of all apples in Attachment 1.
- Problem 4: Based on Attachment 1, calculate the two-dimensional area of each apple with the lower left corner of the photo as the coordinate origin and estimate its respective masses, use a histogram to illustrate the mass distribution of all apples in Attachment 1.
- Problem 5: Based on the features of harvest-ready apples' images in Attachment 2, train a model to recognize the apples in Attachment 3, and draw the relevant distribution histogram of the ID numbers of all apples in Attachment 3.

1.3 Our Work

Questions 1 and 2 require us to count and locate the apples. This article first performs data processing such as light removal, Gaussian filtering and dilation and so on to the images in

Attachment 1, removing most of the noise, extracting the main content of the image, and outlining the figure using B-spline curves. By circling the apple to count and locate, we draw the required histograms and scatter plots.

Question 3 requires us to estimate the maturity status of apples. We obtain more training samples by cropping, flipping and rotating, scaling, shifting, Gaussian noise, and color jitter, construct a Convolutional Neural Network (CNN) model, use ReLU function to introduce nonlinear transformation for activation, and then use MaxPooling algorithm to reduce spatial dimension by retaining the maximum value of each region. Among them, in order to reduce the model's excessive dependence on certain specific neurons, we also constructed a dropout layer to randomly discard neurons, effectively preventing overfitting.

Question 4 requires us to calculate the two-dimensional area of apples and estimate its mass. We first generate points randomly using the Mason rotation algorithm, and then use ray crossing algorithm and Monte Carlo algorithm to determine whether the random points are within the contour of the apple. The probability density function of the random points can be used to estimate the two-dimensional area of the apple. Subsequently, by examining the relationship between the number of apples in the image and the two-dimensional area of apples, the quality of apples can be estimated consulting relevant data on apple density and a histogram of apple quality distribution can be drawn.

Question 5 requires us to extract fruit image features from Attachment 2 and train a model to recognize the fruits in Attachment 3. We first preprocess the images in Attachment 2, improving the training performance of the model through steps such as size adjustment and normalization. At the same time, to solve the gradient problem caused by increasing learning depth, we propose an innovative approach of introducing residual blocks, allowing the network to learn residual mapping. Finally, after epochs convergence, the apple in Attachment 3 is identified and its ID distribution histogram is drawn.

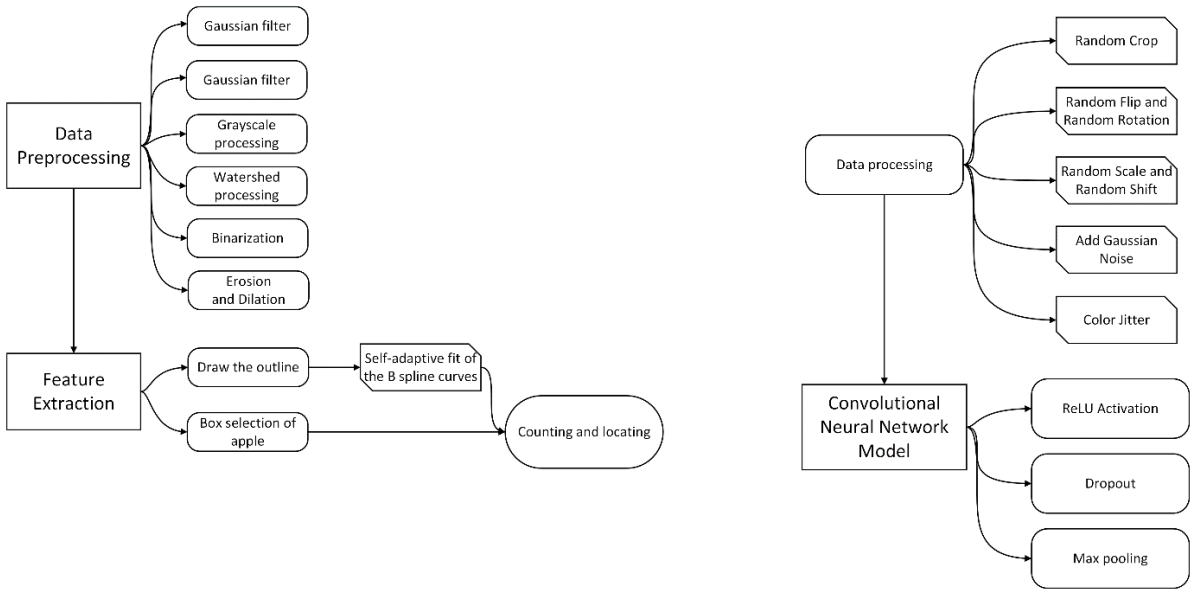


Figure 1: flow diagram of our work

2 Assumptions and Justifications

To simplify the model, we have made the following reasonable assumptions.

• Assumption 1: The maturity of apples in one image is the same.

Justification: The image of attachment 1 is mostly an image of an apple tree or an apple orchard. In one image, the orientation of the apples on the apple tree is almost the same. In similar geographical environments, the nutrients of the light and rainwater received by the apples are basically the same, and the difference in maturity is not very significant. To improve operational efficiency, this article defaults the apples in one image to one maturity.

• Assumption 2: Apple is a sphere with uniform density

Justification: After consulting relevant materials, it was found that the average density of apples is about 0.9g/cm3, and most apples appear as spheres. To facilitate calculation and better deduce the quality of apples from two-dimensional areas, we have made this assumption.

3 Notations

The key mathematical notations used in this paper are listed in Table 1.

Table 1: Notations used in this paper

Symbol	Description
G(x,y)	the distance from the origin of the Fourier transform
B	Convolution template
U	Real number sequence
S	The two dimensional area of apple

4 Apple count and location based on OpenCV

4.1 Data Description

Attachment 1 contains 200 images related to apples. Due to the low clarity and cluttered information of various images, we first preprocessed the images to minimize the influence of apple branches, leaves, and other disrupting background information.

(a) inhomogeneous lighting (b) leaf occlusion (c) fruit occlusion

Figure 2: Image of apples under different environments

4.1.1 Light processing

During the image acquisition process, the influence of geographical environment and oc-

clusion between objects may lead to inhomogeneous lighting of the image, making some important details unable to be highlighted or even masked, which is adverse to the accuracy of apple recognition. In this article, the image is first subjected to lighting removal treatment to eliminate the impact of uneven lighting on the image.

Firstly, normalize the input image and adjust the brightness based on the brightness gain and intermediate value, limiting the pixel values of the image within the range of [0, 1] and ensuring that the lighting range is between [-100, 100]. Determine the maximum value of pixels R, G, and B is denoted as max, the calculation formula for lightness and brightness is as follows:

$$Lightness = \frac{max}{255} \times 100\% \tag{1}$$

$$Brightness = 30\%R + 59\%G + 11\%B \tag{2}$$

Convert the image to a grayscale image, normalize it, and adjust the lightness based on the lightness gain and median value. Then, output the optimized adjusted brightness image to obtain the following image:

Figure 3: figure after light processing

4.1.2 Gaussian filter

The color images provided in Attachment 1 are all 180×270 pixels. Due to the changes in different environments, most of the images are degraded and contain a lot of noise information, which is not beneficial to the recognition of the target apple and will reduce the accuracy of detection. Therefore, we use Gaussian filtering to smooth the given image, blur it, and eliminate noise to better detect and recognize features in the image. The Gaussian distribution function of the filter used in this article is as follows:

$$G(x,y) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2+y^2}{2\sigma^2}} \tag{3}$$

Among them, σ is the standard deviation of the Gaussian curve, $G(x,y)$ is the distance from the origin of the Fourier transform.

After convolving it by function, the following image can be obtained:

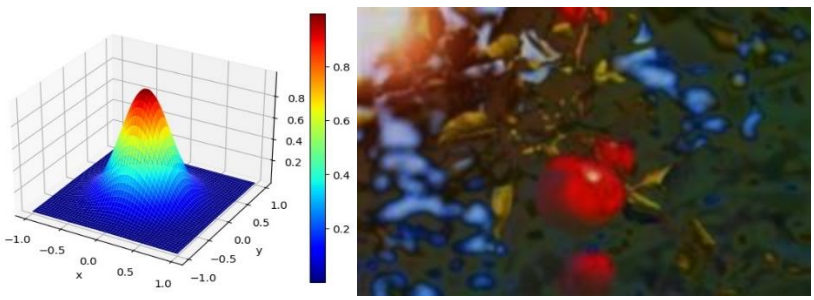


Figure 4: figure of Gaussian function and the figure after Gaussian processing

4.1.3 Grayscale processing

After completing the Gaussian filtering process, in order to visually enhance contrast, highlight the target area, as well as making the operation faster, we use the weighted average method to perform grayscale processing on the image, transforming each pixel from storing RGB in three bytes to storing grayscale values in one byte. When the grayscale value is 255, it is the brightest (pure white), and when the grayscale is 0, it is the darkest (pure black).

Since the object of study in this article is apples, with red as its main color, the weight of RGB is continuously adjusted based on this. After continuous parameter adjustment, the following formula is used to weight and average the RGB components:

$$\mathrm{Gray}(i,j) = 0.299\mathrm{R}(i,j) + 0.587\mathrm{G}(i,j) + 0.114\mathrm{B}(i,j) \tag{4}$$

Therefore, we obtained the following figure:

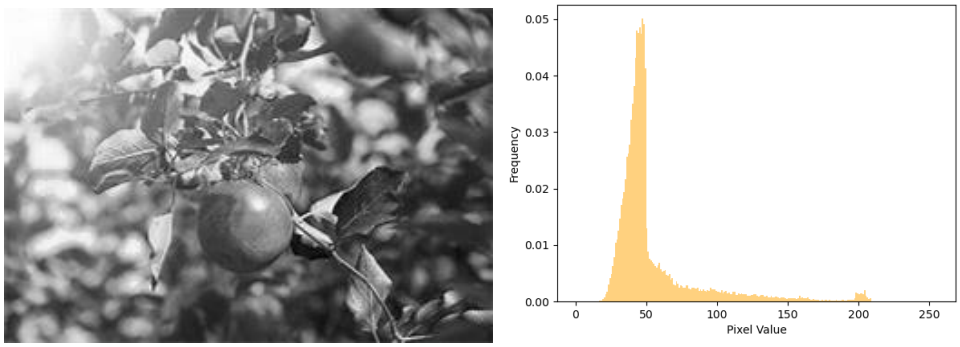


Figure 5: grayscale figure and its collation map

4.1.4 Watershed Model

When there are a large number of apples in the image, overlapping and obstructing apples can cause the main body of the image to be too concentrated, resulting in a deviation in counting. To achieve more accurate apple counting and separately count partially occluded apples, this paper adopts a watershed algorithm to reduce the difference in counting caused by mutual occlusion between apples.

The grayscale value of each pixel in the image represents the altitude of the point, and each local minimum is called the catchment basin of the region. The model is slowly immersed in water, and as the immersion deepens, the influence range of each local minimum gradually expands. Finally, the boundary of the influence of adjacent minimum values forms a watershed.

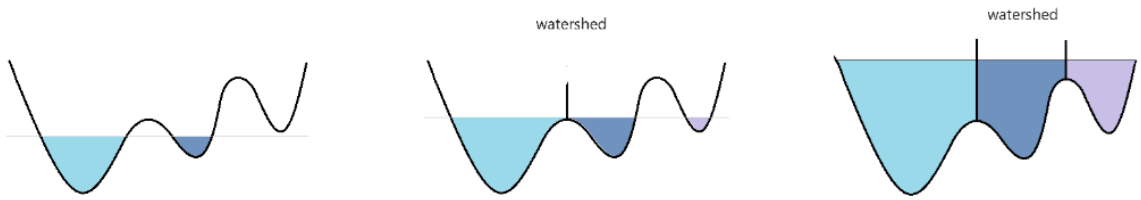


Figure 6: sketch map of watershed

In addition, to avoid excessive segmentation caused by small changes in grayscale values, we perform threshold processing on gradient images, and the formula is as follows:

$$G(x,y) = \max\left(grad\left(f(x,y)\right), g_\theta\right) \tag{5}$$

Among them, g_θ represents the threshold.

4.1.5 Binarization

To enhance the visual effect, we use binarization processing to segment the image, transforming the obtained grayscale image into a binary image. We use the iterative method to calculate the threshold, and obtains the optimal threshold through multiple rounds of iterative operations.

Set the initial threshold to T_0, when pixel values are greater than or equal to T_0, label the pixels as foreground regions, while pixels with a value less than T_0 are labeled as background regions. Calculate the average pixel value for both the background and foreground regions, respectively m_1, m_2, and take half of the sum of m_1 and m_2 as the threshold for the next iteration, until the difference between the current iteration threshold and the previous iteration threshold is less than 0.1, and the iteration ends.

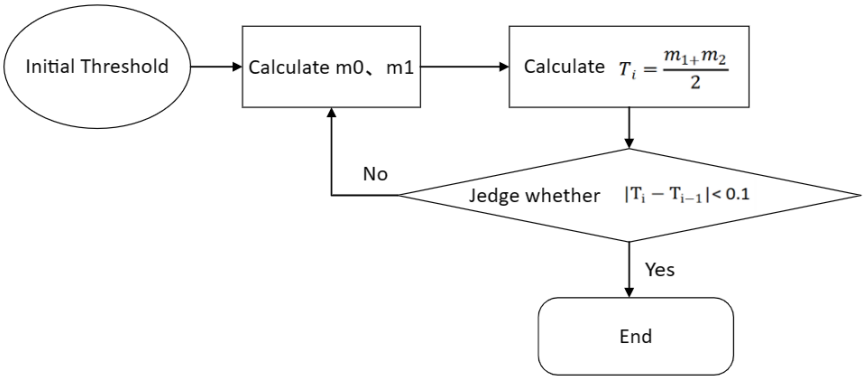


Figure 7: the flow diagram of calculating threshold

When the pixel grayscale value is greater than or equal to this value, the pixel value is set to 1, otherwise it is set to 0. Finally, the following images were obtained:

4.1.6 Erosion and dilation

Considering the presence of many small white spots around the main image, this article uses erosion to remove most of the noise and dilates the eroded image to expand the main target area in the image. By adjusting the contour shape through magnetic attraction, the image processing is completed.

Erosion: We construct a 3x3 convolutional template B to corrode the binarized image A, using the following formula:

$$A \ominus B = \{a | B + a \subseteq A\} \tag{6}$$

Among them, a is an element in set A. The formula can obtain the set of origin positions of B when B is completely included in A. The following diagram is a simple corrosion diagram:

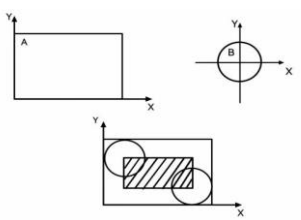


Figure 8: the diagrammatic sketch for corrosion

Dilation: Dilation can be seen as a dual operation of erosion. The convolutional template B is continuously moving. If B is included in the range of A, then this point is recorded. The formula is as follows:

$$A \oplus B = \{a | B + a \cup a \neq \emptyset\} \tag{7}$$

After multiple rounds of erosion and dilation, the following image that is more precise to target recognition is obtained:

4.2 Solution to Question 1

Question 1 requires calculating the number of apples in each image. To facilitate counting, we first perform edge detection on the image and outline the main body of the image.

4.2.1 Outline by self-adaptive B-spline curve

In order to better outline the apple, we did not choose to use regular shapes to outline the shape of the apple. Instead, we proposed a B-spline curve and surface adaptive fitting algorithm based on non-uniform and irrational B-spline fitting method to meet the requirements of different fitting accuracies.

Let $U = U_0, U_1, \ldots, U_m$ be a monotonically non-decreasing real number sequence, $u_i (i = 0,1, \ldots, m-1)$ is called a node, U is called a node vector, denoted by $N_{i,p}(u)$ represents the (p+1) order B-spline basis function, and the formula is as follows:

$$N\_(i,0)\ (u) = \begin{cases} 1, u_i \leq u \leq u_{i+1} \\ 0, otherwise \end{cases} \tag{8}$$

$$N\_(i,p)\ (u) = \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u) \tag{9}$$

Next, perform global surface interpolation on the given data points, taking $(n+1) \times (m+1)$ points $Q_{k,l}, k = 0,1,\ldots,n;\ l = 0,1,\ldots,m$, with its relative parameter values $(\overline{u_k}, \overline{v_l})$, create a irrational (p,q) degree B-spline surface:

$$Q_{k,l} = S(\overline{u_k}, \overline{v_l}) = \sum_{i=0}^{n} \sum_{l=0}^{m} N_{i,p}(\overline{u_k}) N_{j,p}(\overline{v_l}) P_{i,j} \tag{10}$$

After completing the interpolation, the image is approximated to construct a curve C that approximates the given data points.

$$C(u) = \sum_{i=0}^{n} N_{i,p}(u) P_i, \qquad u \in [0,1] \tag{11}$$

Among them, $Q_0 = C(0), Q_m = C(1)$, let

$$R_k = Q_k - N_{0,p}(\overline{u_k}) Q_0 - N_{n,p}(\overline{u_k}) Q_m, \qquad k = 1, \ldots, m-1 \tag{12}$$

$$f = \sum_{k=1}^{m-1} |Q_k - C(\overline{u_k})|^2$$

$$= \sum_{k=1}^{m-1} [H_k \cdot H_k - 2 \sum_{i=1}^{n-1} N_{i,p}(\overline{u_k})(H_k \cdot H_i) \tag{13}$$

$$+ (\sum_{i=1}^{n-1} N_{i,p}(\overline{u_k})P_i)(\sum_{i=1}^{n-1} N_{i,p}(\overline{u_k})P_i)]$$

Thus we obtain a linear system of equations with $n-1$ unknowns and $n-1$ equations

$$(N^T N)P = H \tag{14}$$

Among them, N is a $(m-1) \times (n-1)$ matrix made of scalar

$$N = \begin{bmatrix} N_{1,p}(\overline{u_1}) & \dots & N_{n-1,p}(\overline{u_1}) \\ N_{1,p}(\overline{u_{m-1}}) & \dots & N_{n-1,p}(\overline{u_{m-1}}) \end{bmatrix} \tag{15}$$

$$H = \begin{bmatrix} N_{1,p}(\overline{u_1})H_1 + \dots + N_{1,p}(\overline{u_1})H_{m-1} \\ N_{n-1,p}(\overline{u_{m-1}})H_1 + \dots + N_{n-1,p}(\overline{u_{m-1}})H_{m-1} \end{bmatrix} \cdots \tag{16}$$

After obtaining the B-spline curve, reverse calculate the control vertex d of the B-spline fitting curve obtained based on known data points_ Define a cubic B-spline fitting curve with $d_i(i = 0,1,\dots,n)$ and the node vector $U = [u_0, u_1, \dots, u_{n+k+1}]$ obtained by parameterizing the accumulated chord length. The curve equation is expressed as follows:

$$p(u) = \sum_{i=0}^{n} d_i N_{i,3}(u) \tag{17}$$

By continuously adjusting its offset, we can obtain：

$$p \ast= \sum_{i \neq j=0}^{n} d_i N_{i,3}(u) + (d_j + \varphi r)\, N_{j,3}(u) \tag{18}$$

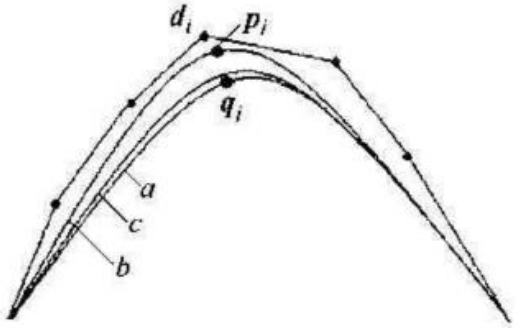


Figure 9: self-adaption B-spline curve

4.2.2 Apple count

After outlining the object with a contour, select the smallest circle and calculate the number of circles to determine the number of apples in the image. Export the results of apple quantity and conduct descriptive statistical analysis on them, resulting in the following table:

Table 1: statistical analysis of apples' number

average	median	mode	minimum	maximum	sum
8.285	6	3	0	50	1657

According to the table, among the 200 images, there are 0 apples with the lowest quantity and 50 apples with the highest quantity. In total, there are 1657 apples in the 200 images. To test the counting effect, we extracted images of fewer apples, more apples, green apples, and flowers to observe their counting situation, and obtained the following figure:

(a) small apple number figure (b) large apple number figure

(c) flower figure (d) green apple figure

Figure 10: counting of apples

From the graph, it can be obtained that the counting situation is relatively ideal. When the number of apples is small, the apples are large and unobstructed, and they contain more feature information, making them easier for the model to accurately detect. When the number of apples is large, due to the small exposed area of the fruits, they contain less effective feature information, making detection more difficult. The model often misses or ignores small apples in the background, which may be related to corrosion operations; The model can accurately identify both green apples and flowers, and the overall effect is good.

By exporting the apple tree of each image through code , a histogram of the distribution of apple numbers can be drawn as follows:

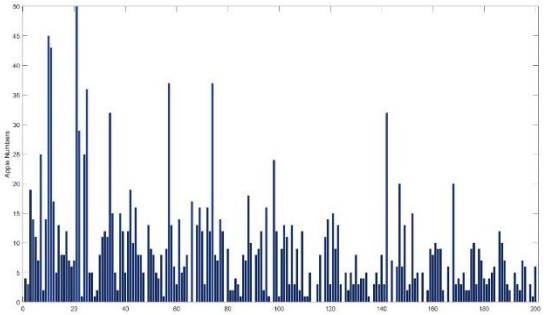


Figure 11: histogram of the distribution of apples

4.3 Solution to Question 2

Establish a coordinate system with the bottom left corner of the image as the coordinate origin. Due to the image pixel size of $180x270$, the bottom boundary of the image is used as the horizontal axis, divided into 270 units, and the left boundary of the image is used as the vertical axis, divided into 180 units. The center point of the circular box is used as its coordinate point, and the coordinates are written around the circular box using code to obtain the apple positioning map

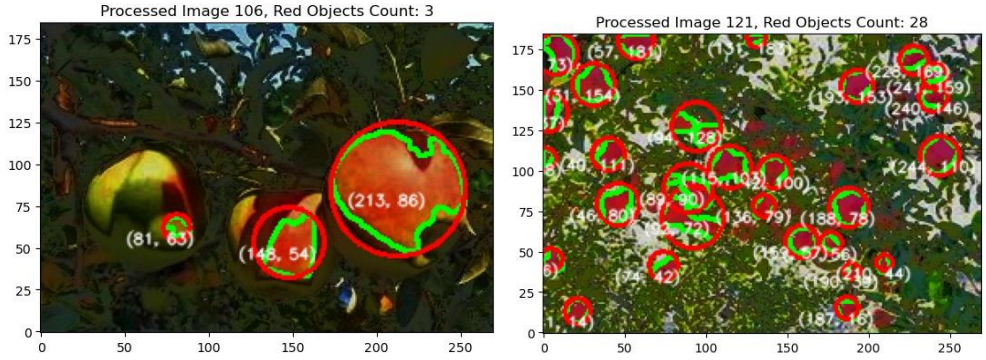


Figure 12: Coordinate graph of apples

The coordinates of apples were statistically plotted on a graph. In order to more intuitively represent the number of apples at different coordinate points, this article also draws their heat maps, and the results are as follows:

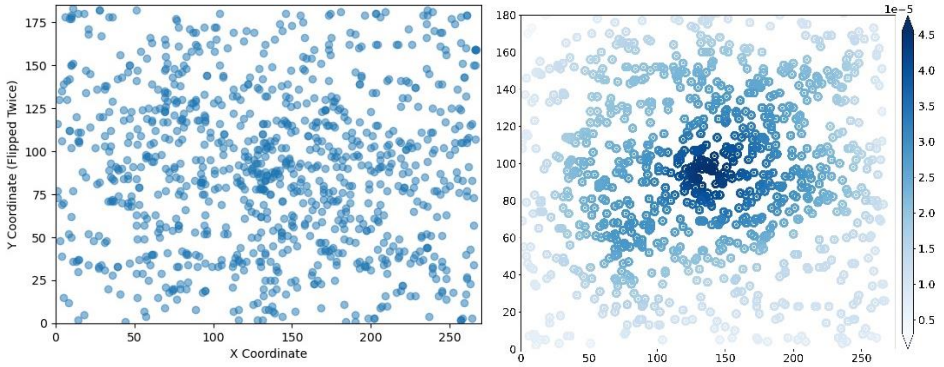


Figure 13: histogram of the position of apples and its heat diagram

5 Check the ripening status of apples based on the stablishment of Convolutional Neural Network Model

5.1 Data processing

5.1.1 Removal of abnormal pictures

To ensure the accuracy of maturity judgment in the model, select and remove images without apples in Attachment 1. There are currently four interference images containing flowers.

5.1.2 Manually classified the apple maturity

Divide maturity into two categories: immature and mature.

5.1.3 Expansion of the image sample data

This article employs a series of operations for data augmentation, including cropping, flipping and rotating, scaling, shifting, Gaussian noise, and color jitter. The purpose of these operations is to generate more training samples and increase the robustness and generalization ability of the model by performing diversity transformations on the original image.

- **Random Crop**:

This article generates regions with a width and height equal to half the original width, height, and length. The cropping operation randomly selects a region in the image, cuts out a specified size image segment, and stores it in a new tuple. This helps to enhance the focus of model learning on different parts, making it more robust. Cropping operations can be used to handle apple parts at different maturity levels, enabling the model to better adapt to inputs of different sizes and shapes.

- **Random Flip and Random Rotation**

The flipping operation horizontally flips the image with a certain probability, while the rotation operation randomly rotates the image at a certain angle. This helps the model learn the features of apples from different perspectives, simulate apple images from different shooting angles and directions, increase the diversity of image data, and make the model more generalizable. When processing, the image rotated by the above operation will have black edges. If you want to remove the black edges of the image, you need to make some sacrifices to the original image and take the maximum inscribed matrix of the rotated image, which has the same aspect ratio as the original image. To calculate the coordinates Q of the tangent matrix, it is necessary to obtain the rotation angle and the edge length OP of the original image matrix, as shown in the following figure:

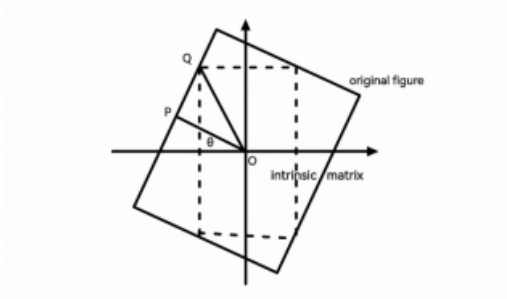


Figure 14: figure of random rotation

The final calculation formula is as follows:

$$k = \frac{\cos\theta + \sin\theta \tan\theta}{r \tan\theta + 1} \tag{19}$$

$$r = \begin{cases} \frac{h}{w}, & h > w \\ \frac{w}{h}, & otherwise \end{cases} \tag{20}$$

$$w' = kw, h' = kh \tag{21}$$

• **Random Scale and Random Shift**

The scaling operation simulates apple images of different distances and scales by randomly adjusting the size of the image. The shift operation introduces positional changes by translating the image. This is very helpful for model learning to adapt to input images of different resolutions, helping the model learn to recognize apples at different distances, sizes, and positions in the image.

• **Add Gaussian Noise**

Gaussian noise operation simulates the noise environment in the real world by adding Gaussian noise to the image, improving the model's adaptability to noisy environments. The density of Gaussian noise depends on the formula $G(x, sigma)$, where X represents the mean and sigma represents the standard deviation. For each input pixel Pin, the output pixel Pout is obtained Where d is a linear random number, and $G\ (d)$ is the Gaussian distribution random value of the random number. The following is the Gaussian sampling distribution formula $G\ (d)$:

$$\text{Pout} = \text{Pin} + \text{XMeans} + \text{sigma} * \text{G(d)} \tag{22}$$

• **Color Jitter**

The color jitter operation increases the color variation of the apple image by adjusting the brightness, contrast, and saturation of the image. This improves the model's ability to adapt to apple features under different lighting and color conditions, and enhances its ability to adapt to real-world scene changes.

5.1.4 Normalization of image data

Before model training, we preprocessed the input data. Load the CIFAR-10 dataset and normalize the images by scaling pixel values to between 0 and 1. This helps to accelerate the convergence of the model and improve the stability of training.

5.2 The establishment of convolutional neural network (CNN) model

In response to Question 3, this article constructs a CNN convolutional neural network model to handle the binary classification problem of judging apple maturity. The convolutional neural network structure designed in this article includes two convolutional layers, two max pooling layers, two Dropout layers, and two fully connected layers. The principle of each step is as follows:

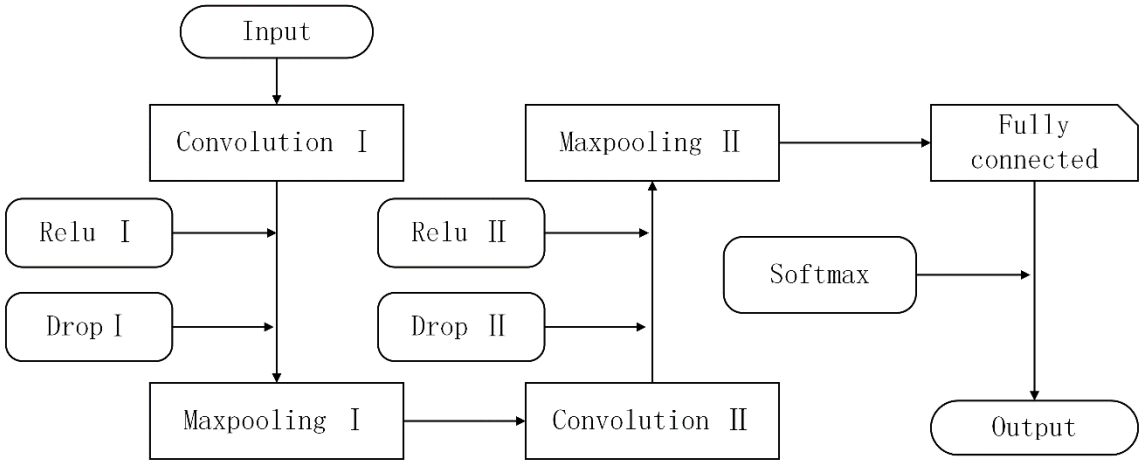


Figure 15: flow diagram of CNN

5.2.1 Construction of Convolutional Layers

The main role of convolutional layers in convolutional neural network models is to extract features from images. Convolution operation is a mathematical operation that involves sliding a convolution kernel to perform element by element inner product calculation and accumulation on input data, and then adding up all product results to obtain a new pixel value at the corresponding position. This process is carried out on the entire input data, forming an element of the output feature map.

In order to increase and balance the weights of central and edge data, and maintain the size of the output feature map, this paper adopts the "same" filling mode, which automatically selects the appropriate number of filling circles based on the size of the kernel to keep the input and output sizes the same.

In the convolution process, the number of convolutional layers set in this article is 2. The first convolutional layer uses 32 3x3 convolution kernels, and the second convolutional layer uses 64 3x3 convolution kernels. Slide the window over the input data at the default step size (1,1). The following is a schematic diagram of window sliding during the convolution process:

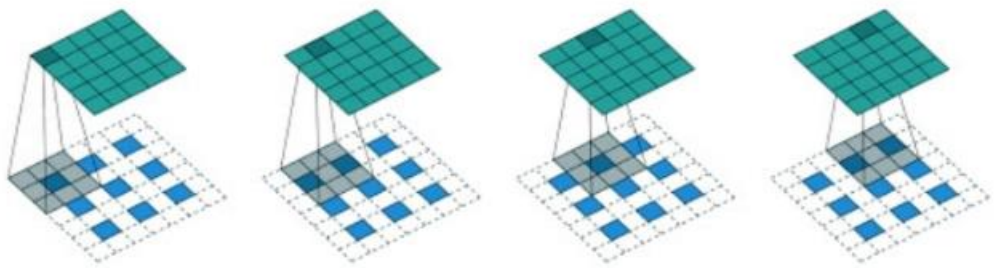


Figure 16: sketch map of convolution

5.2.2 Principle of ReLU activation function

The function of ReLU activation function is to introduce nonlinear transformations, thereby increasing the network's expressive power and ability to learn complex patterns. The principle is: positive channel transmission: for a positive input x, the ReLU activation function maintains the output as x. Negative channel truncation: For a negative input x, ReLU sets its output to zero. By taking the maximum value between input x and zero. The calculation formula for the activation function is as follows:

$$ReLU(x) = \begin{cases} x, & x \geq 0 \\ 0, & otherwise \end{cases} \tag{23}$$

$$ReLU(x) = \begin{cases} 1, & x \geq 0 \\ 0, & otherwise \end{cases} \tag{24}$$

The following is an image of the RuLE activation function and its derivative function:

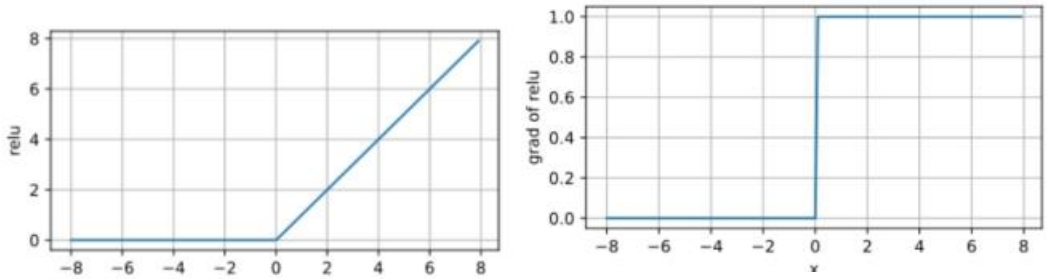


Figure 17: function of ReLU

5.2.3 Dropout Layor

Dropout is a regularization technique in which the Dropout layer is placed between the hidden layers of a neural network. Its main function is to randomly discard some neurons with a certain probability during the training process, thereby reducing the model's excessive dependence on certain specific neurons and helping to prevent overfitting.

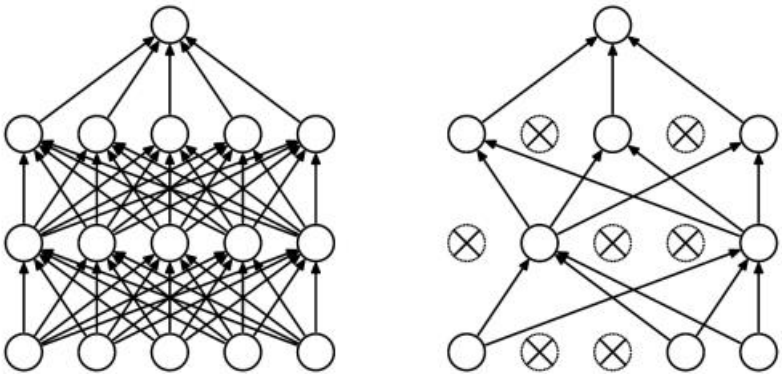


Figure 18: figure of the dropout layer

Minimizing the loss of a Dropout network is equivalent to minimizing a regular network with a regularization term. In this paper, the Dropout loss function is as follows:

$$E_R = \frac{1}{2}(t - \sum_{i=1}^{n} p_i w_i I_i)^2 + \sum_{i=1}^{n} p_i(1 - p_i) {w_i}^2 {I_i}^2 \tag{25}$$

From this formula, it can be concluded that when using Dropout, if the loss rate is 0.5,

Dropout will have the strongest regularization effect. Because $p\,(1-p)$ reaches its maximum value at $p = 0.5$.

Therefore, during the training period, the model in this article sets each neuron to be discarded with a 50% probability. This design makes it possible for each neuron to be excluded during the training process of the network, thereby reducing the complex dependency relationships between neurons and improving the model's generalization ability. However, during the evaluation and inference of the model, the Dropout layer will be turned off, and all neurons will participate in the calculation.

5.2.4 MaxPooling pooling layer downsampling

The MaxPooling algorithm is used in the pooling layer of this model to reduce spatial dimensions by retaining the maximum value of each region. This helps to extract important features from images, reduce computational complexity, and enhance the translation invariance of the model.

5.2.5 Fully connected layer and output layer

Fully connected layer: The fully connected layer highly abstracts and integrates the features extracted by convolutional and pooling layers for higher-level inference and prediction. In this model, the fully connected layer contains 512 neurons, uses ReLU activation function, and includes a Dropout layer to prevent overfitting. The following is a schematic diagram of the fully connected layer principle:

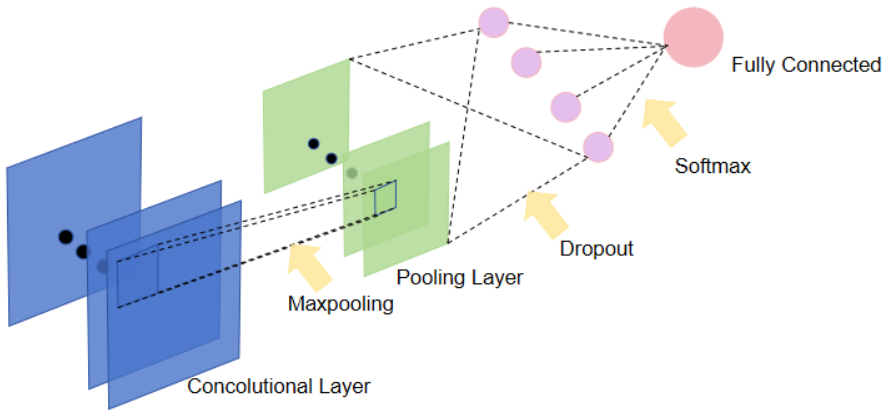


Figure 19: figure of fully connected layer

Output layer: The output layer contains the final output of the model. In this binary classification model, there are two neurons in the output layer, and the softmax activation function is used for probability normalization. Assuming there is an array V, where Vi represents the i-th element in V. The following is the formula for calculating the softmax value of this element:

$$S_i = \frac{e^i}{\sum_j e^j} \tag{26}$$

5.2.6 Model Compilation and Training

This article compiles the model using categorical_crossentropy and Adam optimizer. The

Adam optimizer has the characteristic of adaptive learning rate, which improves the convergence speed of the model through dynamic adjustment during the iteration process. The model was trained on the CIFAR-10 dataset for 20 epochs using a batch size of 16.

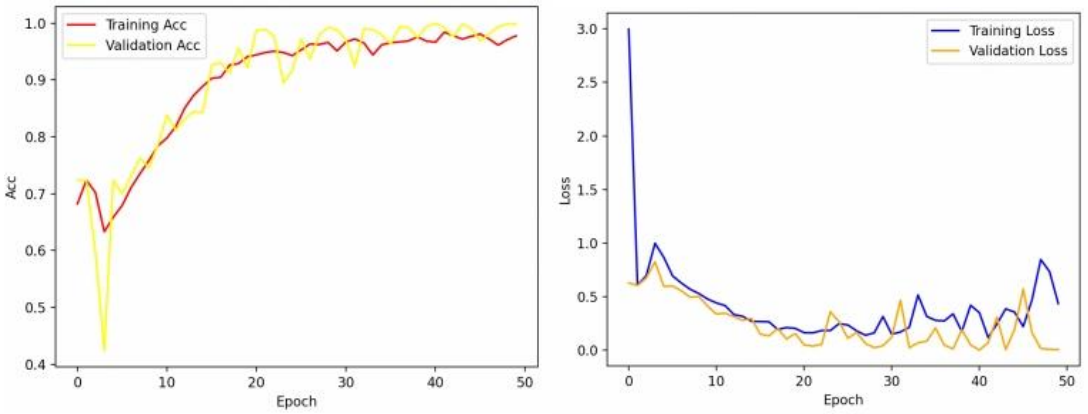


Figure 20:figure of loss function

5.3 Results Display

The prediction results of this binary classification convolutional neural network model are: 54 immature apples, 142 mature apples, and 4 excluded images. The following is a frequency histogram of apple maturity distribution:

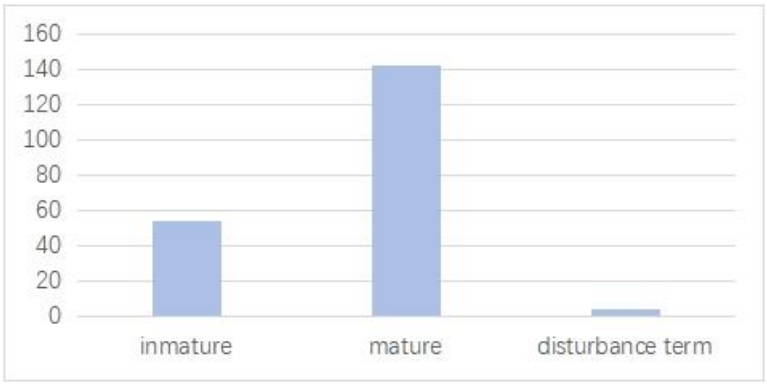


Figure 21: histogram of the maturity distribution of apples

6 Estimation of Apple Quality Based on Calculation of Image Fitting Contour 2D Area

For problem 4, based on the preprocessing of problem 1, we use Mason rotation algorithm to generate random points, and determines whether the random points fall within the apple contour by ray crossover algorithm and Monte Carlo algorithm. By calculating its probability density function and comparing it with the image area, the two-dimensional area of apple can be estimated; Through the relationship between the number of apples in the picture and the two-dimensional area of apples, the actual front view area of apples can be roughly estimated. Based on the study of Apple density and volume, the mass distribution histogram of apples could be obtained.

6.1 Calculation of Apple two-dimensional area by Monte Carlo algorithm

Based on the principle of Monte Carlo, a large number of points are generated randomly to fall on the image given. Then, we record the points falling within the specified Apple frame as, and the points falling outside the frame as, to obtain the two-dimensional areas (in pixels) of the apple as

$$S = \frac{n_1}{n_1 + n_2} \times 180 \times 270 \tag{27}$$

6.1.1 Mason rotation algorithm to obtain random point positions

Mason rotation algorithm uses linear feedback shift register (LFSR) to generate random numbers. The generation of random numbers has a great impact on the calculation of Apple area, in order to avoid the calculation error caused by improper generation of random numbers, this paper uses Mason rotation algorithm to construct a 624 dimensional state vector, where each element is a 32-bit integer to obtain the basic Mason rotation chain. Each time a random number is generated, a series of rotation algorithms would be carried out on the state vector, when some of them will be used to perform XOR, shift, modulo and other operations with other elements to generate the next random number. The formula of Linear Congruence Generator (LCG) applied is as follows:

$$\mathrm{X_{n+1}} = (\mathrm{aX_n} + \mathrm{c}) \bmod \mathrm{m} \tag{28}$$

Among them, X_n is the current generated random number, when X_n is the next random number, m represents modulus, and a represents multiplier, c represents increment, in order to obtain a sequence of random numbers with good randomness and uniformity, and obtain the coordinates of random points in the two-dimensional coordinates of the image. We generated apple images under different random points as shown in the figure:

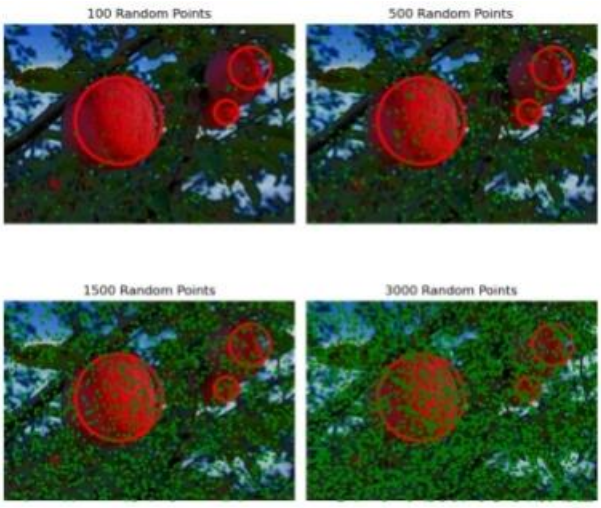


Figure 25: figure of apple images under different random points

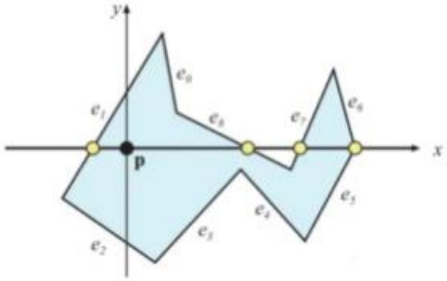
6.1.2 Ray crossing algorithm for determining the position of random points:

We use the ray crossing algorithm to determine whether a random point is inside the fitted

closed circle. The ray polygon intersection algorithm emits a ray, counts the number of intersection points between the ray and the polygon edge, and then determines the position of the points based on the parity of the intersection points. The specific steps are as follows:

Define a ray that intersects with the y-axis of the point to be determined and extends to the right to infinity.

Traverse all edges of a polygon and calculate the intersection point between that edge and the ray. If the intersection point is on the right side of the ray, it is not counted; If on the left side of the ray, count. If the intersection point is on the ray, the number of intersections will be increased by 1, but it will not be counted towards the result. If the number of intersections is odd, the point is inside the polygon, otherwise it is outside the polygon. As shown in the figure:



After being determined by the ray crossing algorithm, the number of random points in the curve is obtained by Monte Carlo simulation, and its two-dimensional area S is finally calculated.

6.2 Two dimensional area rotation estimation of volumetric mass:

Based on the two-dimensional area S of the apple fitting circle obtained by the Monte Carlo algorithm in the previous text, we then perform a three-dimensional rotation on the closed curve to make it a spatial sphere. And use the formula for calculating the spatial volume of a sphere:

$$V = \frac{4}{3}\pi r^3 = \frac{1}{3} r * S \tag{29}$$

Calculate that the volume of the apple is approximately equal to the spatial volume V of the ball, multiply by its pixel ratio, and then multiply by the average density of the apple $\rho \approx 0.9 g/cm^3$ to obtain its final mass m. The following is a histogram of the quality distribution of all apples in Attachment 1:

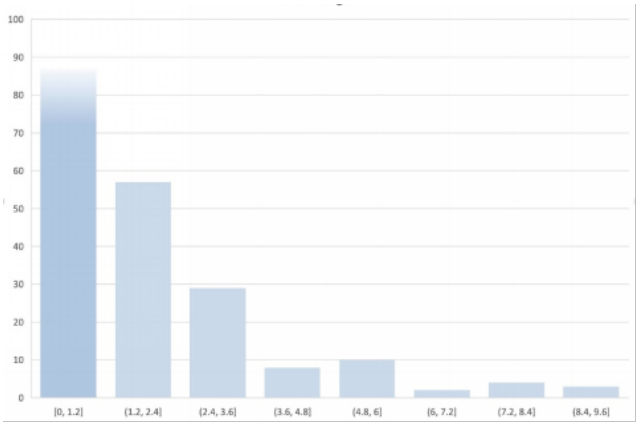


Figure: histogram of the mass distribution of all apples in Attachment 1

7 Building an Apple Neural Network Recognition Model Based on Deep Learning Algorithms

7.1 Training of ResNet-50 Deep Neural Network Model

For Question 5, this article uses the ResNet-50 residual network model from the Convolutional Neural Network (CNN) model. Unlike the convolutional neural network model in problem three, it uses inductive transfer learning and successfully solves the problems of vanishing and exploding gradients in deep neural networks by introducing the concept of residual learning, enabling the network model to train effectively even as the depth increases.

7.1.1 Data processing

This article focuses on resizing, tensor transformation, and normalization of the harvested fruit image dataset provided in Attachment 2. These steps help improve the training effectiveness of the model.

7.1.2 Inductive Transfer Learning

Transfer Learning is a machine learning method that uses the model developed for task A as the initial point and reuses it in the process of developing the model for task B

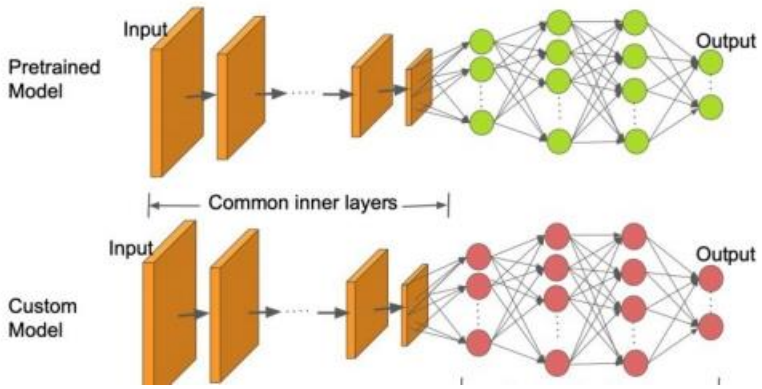


Figure 22: diagram of inductive transfer learning

On the basis of the Resnet pre trained model, a fruit classification recognition model was

constructed through transfer learning. After 30 epochs of training, the model achieved rapid convergence with an accuracy of over 96%.

However, as the number of network layers increases, the problems of vanishing and exploding gradients in the model gradually become prominent. As the number of layers increases, the gradient information gradually decreases during the backpropagation process, making it difficult for the network to converge. Meanwhile, the problem of gradient explosion can also lead to excessive parameter updates in the network, making it difficult to converge normally.

7.1.3 Introducing Residual Block to Handle Network Degradation Problems

In order to solve the problems of vanishing and exploding gradients caused by increasing learning depth, this paper utilizes an innovative approach proposed by ResNet: introducing residual blocks. In traditional feedforward networks, the stacked layers in the network can map the input x to F (x), and the output of this overall network is $H\ (x)$, where $F\ (x) = H\ (x)$. The purpose of residual networks is not to learn the mapping from x to H (x), but rather the difference between x and $H\ (x)$, which is also the origin of the term "residual". Residual $F\ (x) = H\ (x)$ - x, so we try to learn $F\ (x) + x$ instead of directly learning H (x). Its operation is to jump connect the input of a certain layer to the next or even deeper activation layer, and output it together with the output of the current layer through the activation function. As shown in the following figure:

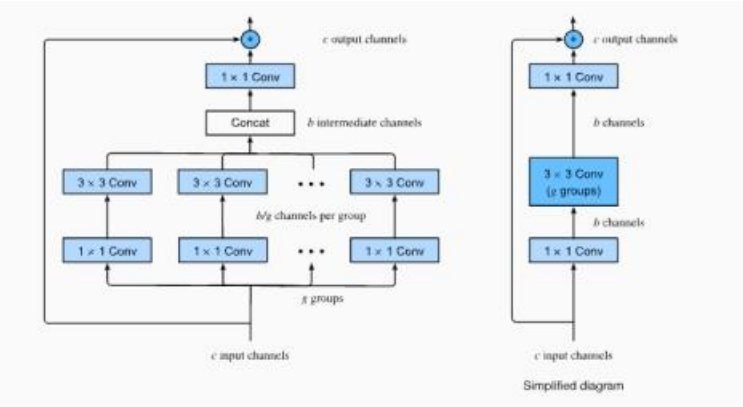


Figure 23: figure of residual block

The design of residual blocks allows the network to learn residual mapping, thereby reducing the problem of gradient vanishing and making the network easier to train.

7.1.4 Loss function and optimizer

Using CrossEntropy Loss as the loss function for classification tasks and Random Gradient Descent (SGD) as the optimizer. Cross entropy can measure the degree of difference between two different probability distributions in the same random variable, representing the difference between the true probability distribution and the predicted probability distribution. The smaller the value of cross entropy, the better the predictive performance of the model. The formula for cross entropy is expressed as:

$$H(p,q) = -\sum_{i=1}^{n} p(x_i)\log(q(x_i)) \tag{30}$$

Random Gradient Descent (SGD) refers to the process in which samples are randomly

shuffled during each iteration to effectively reduce the parameter update cancellation problem caused by samples. Compared to gradient descent method, it is not easy to get stuck in local optimal solutions.

This article starts with the following loss function:$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n$

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{m}(h_\theta(x) - y)^2 \tag{31}$$

According to the following gradient descent method:

$$\begin{aligned}\frac{\partial}{\partial\theta_j}J(\theta) &= \alpha\frac{\partial}{\partial\theta_j}\frac{1}{2}(h_\theta(x) - y)^2 \\ &= 2\cdot\frac{1}{2}(h_\theta(x) - y)\cdot\frac{\partial}{\partial\theta_j}(h_\theta(x) - y) \\ &= (h_\theta(x) - y)x_j\end{aligned} \tag{32}$$

Derive the update formula for the random gradient function: Loop {For i=1 to m, $\{\theta_j := \theta_j + \alpha(y^{(i)} - h_\theta(x^{(i)}))x_j{}^{(i)}$ (for every j)}}

Finally, calculate the accuracy, which is the proportion of correctly predicted samples to the total number of samples.

After 10 epochs, the model began to converge, and the accuracy change curve for 30 epochs is shown below:

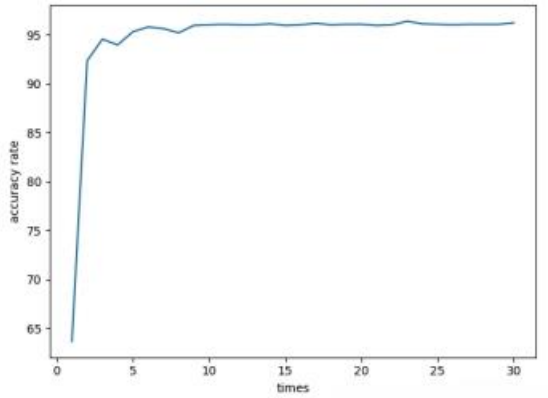


Figure 24: figure of accuracy function

7.2 Apple recognition for Attachment 3 images

By loading the pre trained ResNet-50 model and identifying the apples in Attachment 3, a histogram of the ID distribution recognized by the model as apples is obtained as follows:

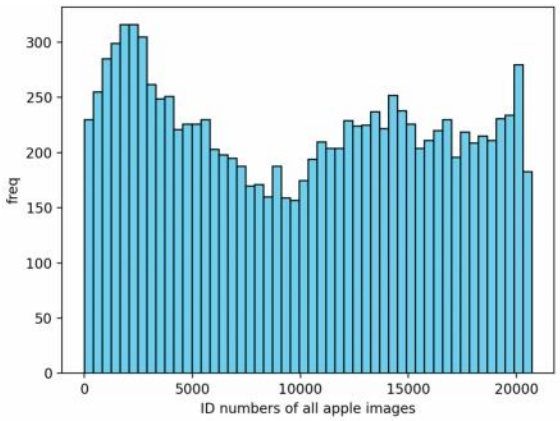


Figure 25: figure of distribution diagram

8 Model Evaluation and Further Discussion

8.1 Strengths

• In the binary image processing, the adaptive threshold is calculated by iterative method, instead of fixing a threshold for all images, so that the selection of threshold is more reasonable.

• B-spline curve adaptive fitting algorithm is used to accurately outline the contour of apples in the processed image.

• Mason rotation algorithm is used to generate random numbers, so that the data has good random distribution. High quality random number generation is more conducive to calculating the two-dimensional area of apples.

• The Relu activation function does not have a saturated region, and there is no gradient disappearance problem to prevent gradient dispersion; The actual convergence speed is faster than sigmoid/tanh; It is more consistent with the biological neural activation mechanism than sigmoid.

8.2 Weaknesses

• Relu activation function neuron death problem: when the weight update of neurons results in negative numbers for all inputs, the gradient would always be zero, and the neurons will no longer learn.

• Dropout layer in a shallow network, the loss rate may be lower than 0.2, the loss of too much input data on the model would be caused by huge loss rate.

•A large number of random samples are required by the calculation of Monte Carlo method, with large amount of calculation and long running time.

8.3 Further Discussion

Improvement: When removing light from the image, a two-dimensional gamma function can be constructed to adaptively correct the scene with different light intensities by adjusting parameters to avoid the interaction of RGB channels.

Leaky relu and parametric Relu activation functions could be used to improve the problems of unstable output distribution, neuron death and network failure that may occur in the

relu function.

9 Conclusion

Our team processed the image through the opencv model, completed the counting and positioning of the apple according to the contour outline and other methods after data preprocessing. After that, CNN convolutional neural network training model is constructed to make a certain judgment on the maturity of the apple. Then we found a method to evaluate the quality of the apple through the Monte Carlo simulation. Based on a large number of literature from academic journals , we finally constructed a model based on a large number of training sets through the images in Attachment 2. This model could significantly distinguish different fruits, and could evaluate other characteristics such as the location and numbers. It could have a wide range of practical significance to improve the labor shortage on the farm. If combined with the research on robots, it has great potential to be put into the apple picking market.

10 References

Xiao, G. (2023) Research on Apple object Recognition in a complex environment based on deep learning. Jiangsu University

Huai, Z. (2023) *Identification of rock debris lithology based on transfer learning*. Journal of the University of Chinese Academy of Sciences 40 (06): 743-750

Yixuan, W. (2023) *Practice teaching of OpenCV machine vision based on project-based teaching*. Computer Knowledge and Technology 19(29):169-171

Merih, L., Ali, C. and Murtaza, C. (2023) *CNN-based automatic modulation recognition for index modulation systems.* Expert Systems With Applications. 240

Appendices

Appendix 1 GaussianFilter.ipynb

Introduce: This code realise the function of Gaussian Filter

```
import cv2
import numpy as np

def myGaussianFilter(src, n, sigma):
    # [1] Initialize
    dst = src.copy()
    # [2] Split channels for color image
    channels = cv2.split(src)
    # [3] Filtering
    # [3-1] Determine Gaussian kernel array
    array = getGaussianArray(n, sigma)
    # [3-2] Apply Gaussian filtering
    for i in range(3):
        gaussian(channels[i], array, n)
    # [4] Merge channels and return
    dst = cv2.merge(channels)
    return dst

def getGaussianArray(arr_size, sigma):
    # [1] Initialize the weight array
    array = np.zeros((arr_size, arr_size))
    # [2] Calculate Gaussian distribution
    center_i, center_j = arr_size // 2, arr_size // 2
    pi = 3.141592653589793
    sum_val = 0.0
    # [2-1] Gaussian function
    for i in range(arr_size):
        for j in range(arr_size):
            array[i, j] = np.exp(-1.0 * ((i - center_i)**2 + (j - center_j)**2) / (2.0 *
sigma**2))
            sum_val += array[i, j]

    # [2-2] Normalize to obtain weights
    array /= sum_val
    return array

def gaussian(src, array, size):
```

```
    temp = src.copy()
    # [1] Scan
    for i in range(src.shape[0]):
        for j in range(src.shape[1]):
            # [2] Ignore edges
            if i > size // 2 - 1 and j > size // 2 - 1 and i < src.shape[0] - size // 2 and j < src.shape[1] - size // 2:
                # [3] Find image input point f(i, j), align with the kernel center
                #     The kernel center is the reference point, the convolution operator => Gaussian matrix rotated 180 degrees for calculation
                #     x, y represent the coordinate of the kernel's weight, i, j represent the coordinate of the image input point
                #     Convolution operator: (f * g)(i, j) = f(i - k, j - l) * g(k, l), f represents the image input, g represents the kernel
                #     Substitute the kernel reference point: (f * g)(i, j) = f(i - (k - ai), j - (l - aj)) * g(k, l), ai, aj are the kernel reference points
                #     Weighted sum, note: the coordinates of the kernel start from the top-left (0, 0)
                sum_val = 0.0
                for k in range(size):
                    for l in range(size):
                        sum_val += src[i - k + size // 2, j - l + size // 2] * array[k, l]
                # Place the intermediate result, the calculated value cannot be mixed with the uncalculated value
                temp[i, j] = sum_val

    # Place in the original image
    src[:, :] = temp

# Example usage
src = cv2.imread("Attachment_2/1.jpg")

# Pass source image, filter size, and sigma to the myGaussianFilter function
result = myGaussianFilter(src, 5, 1.5)

# Display the result

#cv2.imshow("Result", result)
#cv2.waitKey(0)
#cv2.destroyAllWindows()
```

Appendix 2 Watershed.ipynb

Introduce: This code uses watershed to help distinguish Overlapping apples

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
def ReadImg(image_path):
    img = cv2.imread(image_path, 1)
    return img

# Gaussian blur
def GausBlur(src):
    dst = cv2.GaussianBlur(src, (5, 5), 1.5)
    return dst

# Opening morphology (alternative to open_mor)
def open_mor(img):
    kernel = np.ones((5, 5), np.uint8)
    result = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
    return result

# Detect red objects
def detect_red_objects(src):
    # Set red threshold range
    lower_red = np.array([0, 0, 100])
    upper_red = np.array([100, 100, 255])

    # Create a mask based on the threshold
    mask = cv2.inRange(src, lower_red, upper_red)

    # Extract red objects using the mask
    red_objects = cv2.bitwise_and(src, src, mask=mask)

    return red_objects

# Contour fitting
def draw_shape(open_img, src, markers):
```

```
    contours, hierarchy = cv2.findContours(open_img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    apple_count = 0
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if area > 50:    # Assume the minimum area of a red object is 100
            # Get the center and radius
            (x, y), radius = cv2.minEnclosingCircle(cnt)
            center = (int(x), int(y))
            radius = int(radius)

            # Draw a circle
            cv2.circle(src, center, radius, (0, 0, 255), 2)

            apple_count += 1

    return src, apple_count

# Test watershed algorithm
def test_watershed(image):
    image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    _, thresh = cv2.threshold(image_gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

    kernel = np.ones((3, 3), dtype=np.uint8)
    opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)

    sure_bg = cv2.dilate(opening, kernel, iterations=2)

    dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
    cv2.normalize(dist_transform, dist_transform, 0, 1.0, cv2.NORM_MINMAX)
    _, sure_fg = cv2.threshold(dist_transform, 0.5 * dist_transform.max(), 255, 0)
    sure_fg = np.uint8(sure_fg)

    unknown = cv2.subtract(sure_bg, sure_fg)

    _, markers = cv2.connectedComponents(sure_fg)
    markers += 1
    markers[unknown == 255] = 0
    markers_copy = markers.copy()

    markers = cv2.watershed(image, markers)
```

```
    image[markers == -1] = [0, 0, 255]

    return image

# Process all images
for i in range(1, 6):    # Assuming image numbers from 1 to 5
    image_path = f'Attachment_2/{i}_processed.jpg'
    src = ReadImg(image_path)
    gaus_img = GausBlur(src)
    red_objects_img = detect_red_objects(gaus_img)
    gray_img = cv2.cvtColor(red_objects_img, cv2.COLOR_BGR2GRAY)
    _, thres_img = cv2.threshold(gray_img, 1, 255, cv2.THRESH_BINARY)
    open_img = open_mor(thres_img)

    sure_bg = cv2.dilate(open_img, None, iterations=3)
    dist_transform = cv2.distanceTransform(open_img, cv2.DIST_L2, 5)
    _, sure_fg = cv2.threshold(dist_transform, 0.7 * dist_transform.max(), 255, 0)
    sure_fg = np.uint8(sure_fg)
    unknown = cv2.subtract(sure_bg, sure_fg)
    _, markers = cv2.connectedComponents(sure_fg)
    markers = markers + 1
    markers[unknown == 255] = 0
    markers = cv2.watershed(src, markers)

    src[markers == -1] = [0, 0, 255]

    result_img, apple_count = draw_shape(open_img, src, markers)

    # Apply watershed to the result
    result_with_watershed = test_watershed(result_img)

    # Display the processed image with watershed result
    plt.imshow(cv2.cvtColor(result_with_watershed, cv2.COLOR_BGR2RGB))
    plt.title(f'Processed Image {i}, Red Objects Count: {apple_count}')
    plt.show()
```

Appendix 3 Locations Output.ipynb
Introduce: This code generate the graphs of apples with all the location information

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the image
def ReadImg(image_path):
    img = cv2.imread(image_path, 1)
    return img

# Gaussian blur
def GausBlur(src):
    dst = cv2.GaussianBlur(src, (5, 5), 1.5)
    return dst

# Opening operation (replacement for open_mor)
def open_mor(img):
    kernel = np.ones((5, 5), np.uint8)
    result = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
    return result

# Detect red objects
def detect_red_objects(src):
    # Set red threshold range
    lower_red = np.array([0, 0, 100])
    upper_red = np.array([100, 100, 255])

    # Create a mask based on the threshold
    mask = cv2.inRange(src, lower_red, upper_red)

    # Extract red objects using the mask
    red_objects = cv2.bitwise_and(src, src, mask=mask)

    return red_objects

# Fit contours
def draw_shape(open_img, src):
    contours, hierarchy = cv2.findContours(open_img, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    apple_count = 0
    for cnt in contours:
        area = cv2.contourArea(cnt)
```

```
            if area > 50:    # Assuming the minimum area for red objects is 100
                # Get center and radius
                (x, y), radius = cv2.minEnclosingCircle(cnt)
                center = (int(x), int(y))
                radius = int(radius)

                # Draw circles
                cv2.circle(src, center, radius, (0, 0, 255), 2)

                # Draw coordinates
                cv2.putText(src, f'({center[0]}, {center[1]})', (center[0] - 30, center[1] -
20),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,
cv2.LINE_AA)
                apple_count += 1
                # Print coordinates
                print('Apple coordinates:', x, y)
    return src, apple_count

# Process all images
for i in range(1, 3):    # Assuming image numbers range from 1 to 200
    image_path = f'Attachment_2/{i}_processed.jpg'
    src = ReadImg(image_path)
    gaus_img = GausBlur(src)
    red_objects_img = detect_red_objects(gaus_img)
    gray_img = cv2.cvtColor(red_objects_img, cv2.COLOR_BGR2GRAY)
    _, thres_img = cv2.threshold(gray_img, 1, 255, cv2.THRESH_BINARY)
    open_img = open_mor(thres_img)
    result_img, apple_count = draw_shape(open_img, src)

    # Display images
    plt.imshow(cv2.cvtColor(result_img, cv2.COLOR_BGR2RGB))
    plt.title(f'Processed Image {i}, Red Objects Count: {apple_count}')
    plt.show()
```

Appendix 4 Erode_and_Dilate.ipynb
Introduce: 这里放上附录 2 的介绍

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Read the image
image = cv2.imread('/Users/zhangying/iCloud Drive (Archive) - 2/Desktop/Andrea/Academic 学术/建模资料/亚太地区 A 题解题/Attachment_3/2_result.jpg', cv2.IMREAD_GRAYSCALE)

# Check if the image is loaded successfully
if image is None:
    print("Error: Unable to read the image.")
else:
    # Define the kernels for erosion and dilation (adjust size and shape as needed)
    kernel_erode = np.ones((5, 5), np.uint8)
    kernel_dilate = np.ones((5, 5), np.uint8)

    # Perform erosion operation
    image_eroded = cv2.erode(image, kernel_erode, iterations=1)

    # Perform dilation operation
    image_dilated = cv2.dilate(image, kernel_dilate, iterations=1)

    # Display the original image, eroded result, and dilated result
    plt.subplot(131), plt.imshow(image, cmap='gray'), plt.title('Original Image')
    plt.subplot(132), plt.imshow(image_eroded, cmap='gray'), plt.title('Eroded Image')
    plt.subplot(133), plt.imshow(image_dilated, cmap='gray'), plt.title('Dilated Image')

    plt.show()

import cv2
import numpy as np
from matplotlib import pyplot as plt
import os
from hashlib import sha256

# Read the image in color
image = cv2.imread('/Users/zhangying/iCloud Drive (Archive) - 2/Desktop/Andrea/Academic 学术/建模资料/亚太地区 A 题解题/Attachment_3/5_result.jpg', cv2.IMREAD_COLOR)

# Check if the image is successfully loaded
```

```
if image is None:
    print("Error: Unable to read the image.")
else:
    # Split the image into its color channels
    b, g, r = cv2.split(image)

    # Define erosion and dilation kernels
    kernel_erode = np.ones((5, 5), np.uint8)
    kernel_dilate = np.ones((5, 5), np.uint8)

    # Apply erosion and dilation to each color channel
    b_eroded = cv2.erode(b, kernel_erode, iterations=1)
    g_eroded = cv2.erode(g, kernel_erode, iterations=1)
    r_eroded = cv2.erode(r, kernel_erode, iterations=1)

    b_dilated = cv2.dilate(b, kernel_dilate, iterations=1)
    g_dilated = cv2.dilate(g, kernel_dilate, iterations=1)
    r_dilated = cv2.dilate(r, kernel_dilate, iterations=1)

    # Merge the color channels back together
    image_eroded = cv2.merge([b_eroded, g_eroded, r_eroded])
    image_dilated = cv2.merge([b_dilated, g_dilated, r_dilated])

    # Create a folder to save images
    output_folder = os.path.join('呵呵', sha256(image.tobytes()).hexdigest())
    os.makedirs(output_folder, exist_ok=True)

    # Save images to the folder
    cv2.imwrite(os.path.join(output_folder, 'original_image.jpg'), image)
    cv2.imwrite(os.path.join(output_folder, 'eroded_image.jpg'), image_eroded)
    cv2.imwrite(os.path.join(output_folder, 'dilated_image.jpg'), image_dilated)

    print(f"Images saved to the output folder: {output_folder}")

    # Display the original image, eroded image, and dilated image
    plt.subplot(131), plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB)), plt.ti-
tle('Original Image')
    plt.subplot(132), plt.imshow(cv2.cvtColor(image_eroded,
cv2.COLOR_BGR2RGB)), plt.title('Eroded Image')
    plt.subplot(133), plt.imshow(cv2.cvtColor(image_dilated,
cv2.COLOR_BGR2RGB)), plt.title('Dilated Image')
```

```
    plt.show()
```

Appendix 5 Q3_process_train

Introduce: This code trains the mode based on Attachment 2

```
import numpy as np
import os
import cv2 as cv
import pandas as pd
from PIL import Image, ImageEnhance, ImageFilter
import random

# Function for random cropping of images
def random_crop(image, crop_size):
    # Implementation for random cropping...

# Function for random flipping of images
def random_flip(image):
    # Implementation for random flipping...

# Function for random rotation of images
def random_rotation(image, max_angle):
    # Implementation for random rotation...

# Function for random scaling of images
def random_scale(image, scale_range):
    # Implementation for random scaling...

# Function for random shifting of images
def random_shift(image, shift_range):
    # Implementation for random shifting...

# Function to add Gaussian noise to images
def add_gaussian_noise(image, mean=0, std=25):
    # Implementation for adding Gaussian noise...

# Function for color jitter in images
def color_jitter(image, brightness_factor=0.5, contrast_factor=0.5, saturation_factor=0.5):
    # Implementation for color jitter...
```

```
# Function for image augmentation
def image_augmentation(image_path, save_path):
    # Implementation for various image augmentation techniques...

# Directory to save intermediate augmented images
save_path = ''

# Ensure the directory exists
os.makedirs('hahhaha', exist_ok=True)

# Read an xlsx file
file_path = '成熟度.xlsx'
df = pd.read_excel(file_path)

# Use the replace method to replace values
df['标签'] = df['标签'].replace({'熟': 0, '不熟': 1})

# Show the data size
print(df.shape)

# Convert to dictionary
label = df.set_index('序号').to_dict()['标签']

# Set image width and height
imgwidth = 270
imgheight = 180

# Lists to store image data and tags for training and testing
imgdata_train = []
imgtag_train = []
imgdata_test = []
imgtag_test = []

# Get all files and folders in a specified path
directory = '/Users/zhangying/iCloud Drive (Archive) - 2/Desktop/Andrea/Academic 学
术/建模资料/亚太地区 A 题解题/Attachment 1'
files = os.listdir(directory)

# Iterate through files and folders
for file in files:
    # Get the full path of the file or folder
```

```
        image_path = os.path.join(directory, file)
        if image_path[-3:] == "jpg":
            id = int(image_path.split('/')[-1].split('.')[0])
            print(id)
            # If 'id' is a category
            if id in label.keys():
                # Load image
                img = cv.imread(image_path, 0)
                img = cv.resize(img, (imgheight, imgwidth))
                # Data augmentation
                intermediate_save_path = os.path.join(save_path)
                for i in range(10):
                    img_enhance = image_augmentation(image_path, intermediate_save_path)
                    img_enhance = cv.resize(img_enhance, (imgheight, imgwidth))
                    # Append to training data
                    imgdata_train.append(img_enhance)
                    imgtag_train.append(label[id])
                    # Append to testing data
                    if random.uniform(0, 10) < 3:
                        imgdata_test.append(img_enhance)
                        imgtag_test.append(label[id])

# Convert data lists to numpy arrays and perform reshaping
# Save the processed data
np.save("./x_train.npy", imgdata_train)
np.save("./y_train.npy", imgtag_train)
np.save("./x_test.npy", imgdata_test)
np.save("./y_test.npy", imgtag_test)

import keras
from keras.datasets import cifar10
from keras.models import Sequential
from keras.layers import Dense, Dropout, Activation, Flatten
from keras.layers import Conv2D, MaxPooling2D
import numpy as np

# Binary classification with 2 classes
num_classes = 2
nppath = "./"

# Load data
```

```
x_train = np.load(nppath + "x_train.npy")
x_test = np.load(nppath + "x_test.npy")
y_train = np.load(nppath + "y_train.npy")
y_test = np.load(nppath + "y_test.npy")

x_train = x_train.astype('float32') / 255
x_test = x_test.astype('float32') / 255

# Convert class vectors to binary class matrices.
y_train = keras.utils.to_categorical(y_train, num_classes)
y_test = keras.utils.to_categorical(y_test, num_classes)

model = Sequential()

model.add(Conv2D(32, (3, 3), padding='same', input_shape=x_train.shape[1:]))
model.add(Activation('relu'))

model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))

model.add(Conv2D(64, (3, 3), padding='same'))
model.add(Activation('relu'))

model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))

model.add(Flatten())

model.add(Dense(512))
model.add(Activation('relu'))
model.add(Dropout(0.5))

model.add(Dense(num_classes))
model.add(Activation('softmax'))

model.summary()

# This optimizer is not applicable for this experiment
# opt = keras.optimizers.rmsprop(lr=0.001, decay=1e-6)

# Train the model using Adam optimizer
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
# Training history
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=20, batch_size=16, verbose=0)

# Plot loss and accuracy curves
def plot_loss(history):
    # Extract training and validation loss values
    train_loss = history.history['loss']
    val_loss = history.history['val_loss']

    # Set plot title and labels
    plt.title('Model Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')

    # Plot training and validation loss curves
    plt.plot(train_loss, label='Training Loss', color='blue')
    plt.plot(val_loss, label='Validation Loss', color='orange')

    # Add legend
    plt.legend()

    # Show the plot
    plt.show()

def plot_acc(history):
    # Extract training and validation accuracy values
    train_accuracy = history.history['accuracy']
    val_accuracy = history.history['val_accuracy']

    # Set plot title and labels
    plt.title('Model Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')

    # Plot training and validation accuracy curves
    plt.plot(train_accuracy, label='Training Accuracy', color='red')
    plt.plot(val_accuracy, label='Validation Accuracy', color='yellow')

    # Add legend
    plt.legend()
```

```
    # Show the plot
    plt.show()

# Train the model and record training history
history = model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=20, batch_size=16, verbose=0)

# Plot loss curves
plot_loss(history)

# Plot accuracy curves
plot_acc(history)
```

Appendix 6 Monte Carlo.ipynb

Introduce: This code simulates the Monte Carlo process

```
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image

# Read your own image as the background
custom_background_img = np.array(Image.open("/Users/zhangying/Processed_Image_2.jpg"))

# Create a function to generate an image and add random points
def generate_image(base_image, num_points):
    img = np.copy(base_image)  # Create a copy based on the base image
    height, width = img.shape

    # Generate a specified number of random points on the image
    for _ in range(num_points):
        x, y = np.random.randint(0, width), np.random.randint(0, height)
        img[y, x] = 0  # Add a black point at a random position

    return img

# Generate images with four different numbers of random points
num_points_list = [100, 500, 1500, 3000]
images = [generate_image(custom_background_img, num_points) for num_points in
```

```
num_points_list]

plt.figure(figsize=(8, 8))

for i in range(4):
    plt.subplot(2, 2, i + 1)

    # Get the coordinates of the random points
    coords = np.argwhere(images[i] == 0)

    # Plot a scatter plot with green color for the points
    plt.scatter(coords[:, 1], coords[:, 0], c='green', s=5)  # y-coordinates first, x-coordinates next
    # Display the custom background image
    plt.imshow(custom_background_img, cmap='gray', alpha=1)  # Set alpha value for transparency
    plt.title(f'{num_points_list[i]} Random Points')
    plt.axis('off')  # Turn off the axis

plt.tight_layout()
plt.show()
```

Appendix 7 q5_Train_forecast.ipynb

Introduce: This code is a model trained based on all the data in Attachment 3

```
import torch
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import datetime
import numpy as np

from torchvision.datasets import ImageFolder
from torchvision import transforms

from torchvision.models import resnet50

from sklearn.model_selection import train_test_split
```

```
# 图像变换
transform = transforms.Compose([transforms.Resize((224, 224)),
                                transforms.ToTensor(),
                                transforms.Normalize(
                                    mean=[0.485, 0.456, 0.406],
                                    std=[0.229, 0.224, 0.225]
                                ), ])
# 加载数据集
dataset = ImageFolder('/Users/zhangying/iCloud Drive (Archive) - 2/Desktop/Andrea/Academic 学术/建模资料/亚太地区 A 题解题/Attachment 2', transform=transform)

# 划分训练集与测试集
train_dataset, valid_dataset = train_test_split(dataset, test_size=0.2, random_state=0)

batch_size = 64
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size, shuffle=True, drop_last=True)
test_loader = DataLoader(dataset=valid_dataset, batch_size=batch_size, shuffle=True, drop_last=True)

restnet_pretrained_path = '/Users/zhangying/resnet50-0676ba61.pth'
checkpoint_path = '/Users/zhangying/fruit_reg.pth'
checkpoint_resume = False

if __name__ == "__main__":
    # 加载预训练模型
    model = resnet50()
    model.load_state_dict(torch.load(restnet_pretrained_path))

    # 替换最后一层全连接层,构建新的网络
    num_classes = len(dataset.classes)
    in_features = model.fc.in_features
    model.fc = torch.nn.Linear(in_features, num_classes)

    # 模型训练
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    model.to(device)

    criterion = torch.nn.CrossEntropyLoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
    num_epochs = 1
```

```
    accuracy_rate = []
    for epoch in range(num_epochs):

        print('Epoch [{}/{}], start'.format(epoch + 1, num_epochs))

        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

        print('Epoch [{}/{}], Loss: {:.4f}'.format(epoch + 1, num_epochs, loss.item()))

        # 模型验证
        model.eval()
        correct = 0
        total = 0

        with torch.no_grad():
            for inputs, labels in test_loader:
                inputs, labels = inputs.to(device), labels.to(device)
                outputs = model(inputs)
                _, predicted = torch.max(outputs.data, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        accuracy = correct / total * 100
        accuracy_rate.append(accuracy)
        print('Accuracy: {:.2f}%'.format(accuracy))

    accuracy_rate = np.array(accuracy_rate)
    times = np.linspace(1, num_epochs, num_epochs)
    plt.xlabel('times')
    plt.ylabel('accuracy rate')
    plt.plot(times, accuracy_rate)
    plt.show()

    print(f"{datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')},accu-
racy_rate={accuracy_rate}")
    # torch.save(model.state_dict(), checkpoint_path)
```

```
        torch.save(model,checkpoint_path)

import torch
from torchvision import models, transforms
from torchvision.models import resnet50
from PIL import Image
import os
import re
import matplotlib.pyplot as plt

def extract_numbers_in_brackets(input_string):
        # 使用正则表达式匹配括号内的数字
        pattern = r'\((\d+)\)'
        matches = re.findall(pattern, input_string)

        # 返回匹配到的数字列表
        return [int(match) for match in matches][0]

# 加载预训练的 ResNet-50 模型
model = resnet50()
model_path = './checkpoint/fruit_reg.pth'
# model.load_state_dict(torch.load(model_path))
model = torch.load(model_path)
# print(net)
model.eval()  # 将模型设置为评估模式，这是因为在推理阶段不需要梯度

# 定义数据转换
transform = transforms.Compose([
        transforms.Resize((224, 224)),  # ResNet-50 的输入尺寸为 224x224
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# 获取指定路径下的所有文件和文件夹
directory = '/Users/wuguobin/Documents/model/Attachment/Attachment 3/'
files = os.listdir(directory)

appleID = []
# 遍历文件和文件夹
for file in files:
        # 获取文件或文件夹的完整路径
        image_path = os.path.join(directory, file)
```

```
    # 加载图像并进行预测
    # image_path = '/Users/wuguobin/Documents/model/Attachment/Attachment 3/Fruit (75).jpg'
    image = Image.open(image_path)
    input_tensor = transform(image)
    input_batch = input_tensor.unsqueeze(0)  # 添加一个维度表示批次大小

    # 使用模型进行预测
    with torch.no_grad():
        output = model(input_batch)

    # 加载类标签文件
    # with open('imagenet_classes.txt', 'r') as f:
    #     classes = [line.strip() for line in f.readlines()]

    # 获取预测结果
    _, predicted_idx = torch.max(output, 1)
    # print(predicted_idx)
    # predicted_label = classes[predicted_idx.item()]
    # print(f'Predicted label: {predicted_label}')

    # 如果是苹果，记录苹果 ID
    if predicted_idx==0:
        num = extract_numbers_in_brackets(image_path)
        print(num)
        appleID.append(num)

print(len(appleID))

# 绘制直方图
plt.hist(appleID, bins=50, color='skyblue', edgecolor='black')  # bins 为柱子数量
plt.xlabel('ID numbers of all apple images')
plt.ylabel('freq')
plt.title('distribution histogram')
plt.show()
```